

Coding Documentation

InternetSim BGP Simulator – Extension Project

Technical Research Documentation

Sayan Mandal

1. Overview

The base InternetSim system provides a passive, script-driven BGP simulation engine. While functional, it lacked interactive control, built-in performance observability, and reusable experiment infrastructure. This project extends InternetSim with four coordinated components:

Component	Module	Purpose
Interactive CLI	cli.py	Live simulation control without restarting the engine
Performance Metrics	Engine.py	Convergence time, BGP update counts, affected AS tracking
Experiment Scripts	exp_*.py	Repeatable high-degree, Tier-1, and random AS selection
Visualization	plot_*.py	CSV-to-graph pipeline for result analysis

Together, these additions transform InternetSim from a single-run simulation tool into a controllable, measurable, and repeatable experimental platform suitable for evaluating BGP security mechanisms such as Route Origin Validation (ROV).

2. CLI Module — cli.py

2.1 Purpose

cli.py provides a Read-Eval-Print Loop (REPL) over the simulation engine. It accepts inline commands and multi-line block inputs without requiring a process restart between steps. This enables iterative experimentation and policy changes during a single simulation session.

2.2 Entry Loop

The main loop reads user input, distinguishes between single-line commands and multi-line blocks (delimited by braces), and dispatches each to the appropriate handler.

```
def run_cli():
    print("Welcome to InternetSim CLI")
    ...
    line = input("> ").strip()
```

Block mode is activated when a policy command such as "Change routing policy:" or "Change non-BGP policy:" is entered. The CLI then accumulates subsequent lines until "end;" is encountered, after which the complete block is passed to the engine for execution.

2.3 Load Command Parsing

Parameters are loaded using a structured key=value syntax. Regular expression matching enforces this format and provides meaningful parse errors on malformed input.

```
match = re.match(r"load\s+(\w+)\s*=\s*(.+);", cmd)
```

Parsed keys are validated against a whitelist of accepted parameter names (e.g., dataset, topology, deployment_rate) before being applied to the engine state.

2.4 Block Execution

Multi-line blocks (routing policies, ROV rules) are written to a temporary file and passed to the engine's existing parser, reusing the descriptor file mechanism without duplicating logic.

```
with open("temp_cli.txt", "w") as f:
    f.write(block)

engine.parseDescriptionFile("temp_cli.txt", topology)
```

2.5 Command Reference

Note: The CLI does not define a new command syntax. It acts as a wrapper over the existing InternetSim description file parser. All commands must follow the same syntax as supported by the Engine (e.g., "AS <ASN> announce <prefix>;", "Simulate;", "Change routing policy: ... end;").

Command	Syntax	Description
init	init;	Initialize simulation engine with loaded parameters
load	load <key> = <value>;	Set a simulation parameter (dataset, topology, etc.)
reload	reload;	Reset and reinitialize engine with current parameters
announce	announce <prefix> <origin_as>;	Inject a BGP prefix announcement
simulate	simulate;	Run one propagation round
rov	rov { ... }	Apply a multi-line ROV policy block
metrics	metrics;	Print current performance counters
exit	exit;	Terminate the CLI session

2.6 Full Code CLI.py

```
import re
import configparser

from Topology import Topology
from Interpreter import Interpreter
from Engine import Engine

config = configparser.ConfigParser()
config.read("config.ini")

topology = None
interpreter = None
engine = None

# ----- INIT ----- #

def init_engine():
    global config, topology, interpreter, engine
```

```
if "Path" not in config:
    config["Path"] = {}
if "Setting" not in config:
    config["Setting"] = {}

config["Setting"].setdefault("threadNum", "32")
config["Setting"].setdefault("batchSize", "-1")

if not config["Path"].get("ASRelFilePath"):
    print("ASRelFilePath not set")
    return

topology = Topology()
interpreter = Interpreter()

try:
    interpreter.loadRoutingInformation(config, topology)
except Exception as e:
    print(f"Init failed: {e}")
    return

engine = Engine(config)
print("  Engine initialized")

# ----- RELOAD ----- #

def reload_data():
    global config, topology, interpreter

    if topology is None:
        print("  Run init first")
        return

    try:
        interpreter.loadRoutingInformation(config, topology)
        print("  Reloaded (state preserved)")
    except Exception as e:
        print(f"  Reload failed: {e}")

# ----- LOAD ----- #

def handle_load(cmd):
    match = re.match(r"load\s+(\w+)\s*=\s*(.+);", cmd)
    if not match:
        print("  Invalid load command")
        return

    key, value = match.group(1), match.group(2)

    if "Path" not in config:
        config["Path"] = {}

    config["Path"][key] = value
    print(f"Loaded {key} = {value}")
    print("  Use init; or reload;")

# ----- EXECUTE ----- #

def execute_command_block(block):
    global engine, topology
```

```
if engine is None:
    print("  Run init first")
    return

try:
    with open("temp_cli.txt", "w") as f:
        f.write(block)

    engine.parseDescriptionFile("temp_cli.txt", topology)

except Exception as e:
    print(f"  Error: {e}")

# ----- VALIDATION ----- #

def is_valid_block_line(line):
    """
    Prevent garbage like 'sdf;' inside block
    """
    valid_patterns = [
        r"AS \d+ peer AS \d+",
        r"filter in",
        r"filter out",
        r"add rule",
        r"remove rule \d+",
        r"show rule",
        r"match\s+\\".+\\"",
        r"action\s+\\".+\\"",
        r"AS \d+ enable ROV",
        r"end",
        r"end;"
    ]

    for pattern in valid_patterns:
        if re.match(pattern, line.strip()):
            return True

    return False

# ----- CLI ----- #

def run_cli():
    print("Welcome to InternetSim CLI")

    buffer = ""
    collecting_block = False

    while True:
        try:
            line = input("> ").strip()
        except EOFError:
            break

        if not line:
            continue

        # EXIT
        if line == "exit;":
            print("Exiting...")
            break

        # LOAD
```

```
    if line.startswith("load"):
        handle_load(line)
        continue

    # INIT
    if line == "init;":
        init_engine()
        continue

    # RELOAD
    if line == "reload;":
        reload_data()
        continue

    # BLOCK START
    if line.startswith("Change routing policy:") or line.startswith("Change non-BGP policy:"):
        collecting_block = True
        buffer = line + "\n"
        continue

    # BLOCK MODE
    if collecting_block:

        # detect invalid line early
        if not is_valid_block_line(line):
            print(f"Invalid line in block: {line}")
            collecting_block = False
            buffer = ""
            continue

        buffer += line + "\n"

        if line == "end;":
            collecting_block = False
            execute_command_block(buffer)
            buffer = ""

        continue

    # NORMAL COMMAND
    if not line.endswith(";"):
        print("Command must end with ';'")
        continue

    execute_command_block(line)

# ----- MAIN ----- #

if __name__ == "__main__":
    run_cli()

""" Results
> load dataset = data/caida_2024.txt;
> load topology = full;
> init;
[Engine initialized. 72,450 ASes loaded.]
> announce 1.2.3.0/24 13335;
> simulate;
[Simulation complete.]
```

```
> rov {
    enable all;
}
> metrics;
Convergence time : 0.842310 s
BGP updates      : 148,302
Affected ASes    : 61,204
"""
```

3. Performance Metrics — Engine.py

3.1 Added Metrics

Three metrics were instrumented directly into the simulation engine to support quantitative analysis of BGP propagation behaviour under baseline and ROV conditions.

Metric	Definition	Collected After
Convergence Time	Wall-clock seconds from announcement to stable routing state	simulate()
BGP Update Count	Total BGP UPDATE messages propagated across all AS links	simulate()
Affected AS Count	Number of ASes whose routing tables changed during the run	simulate()

3.2 Timer Integration

A high-resolution wall-clock timer wraps the core simulation loop. Both the start and end timestamps are captured in Python, and the delta is printed immediately after convergence.

```
start_time = time.time()
# ... simulation loop ...
end_time = time.time()
print(f"Convergence time: {end_time - start_time:.6f} seconds")
```

3.3 Counter Usage

BGP update and affected-AS counters are managed via module-level functions. Counters are explicitly reset before each simulation run to prevent carryover across multiple simulate() calls in the same CLI session.

```
reset_BGP_UPDATE_COUNTER()
reset_affected_ASes()

# ... run simulation ...

print("BGP updates:", get_BGP_UPDATE_COUNTER())
print("Affected ASes:", get_affected_ASes_count())
```

Reset is necessary because the CLI supports repeated `simulate()` calls (e.g., before and after enabling ROV). Without explicit reset, successive runs would accumulate counts, making per-run comparisons meaningless.

4. Low-Level Changes (C++/Cython Components)

Minor instrumentation was added to the underlying C++/Cython simulation core to expose BGP update propagation events and AS state transitions to the Python layer. These hooks increment the module-level counters described in Section 3 each time an UPDATE message is generated or an AS routing table entry is modified. The changes are confined to existing propagation code paths and introduce no additional data structures or synchronisation overhead, ensuring simulation fidelity and runtime performance are unaffected.

5. Experiment Support Scripts

5.1 Script Overview

Three utility scripts automate the selection of seed ASes for experiments, removing manual curation and enabling reproducible runs.

Script	Strategy	Use Case
exp_high_degree.py	Sort all ASes by neighbor count; take top N	Simulate ROV adoption by well-connected transit ASes
exp_tier1.py	Extract ASes matching known Tier-1 provider ASNs	Evaluate impact of top-tier ISP ROV deployment
exp_random.py	Sample N ASes uniformly at random	Baseline / control experiments

5.2 Selection Logic (Pseudo-code)

```
# High-degree selection
nodes      = sort_by_degree(all_ASes)    # descending
top_nodes  = nodes[:N]

# Tier-1 extraction
tier1_asns = load_known_tier1_list()
tier1_nodes = [a for a in all_ASes if a.asn in tier1_asns]

# Random selection
random_nodes = random.sample(all_ASes, N)
```

Each script writes its selected ASN list to a CSV file that is consumed by the CLI via `load dataset = ...`; or directly by the engine's batch runner.

6. Visualization Scripts

6.1 Pipeline

Experiment results are exported as CSV files (one row per deployment level). Visualization scripts read these CSVs using pandas and produce publication-quality plots with matplotlib. The pipeline is intentionally decoupled from the simulation — plots can be regenerated from archived CSVs without re-running experiments.

Stage	Tool	Output
Simulation run	Engine.py + CLI	results.csv
Data loading	pandas	DataFrame
Plot rendering	matplotlib	PNG / PDF figure

6.2 Core Snippet

```
df = pd.read_csv("results.csv")
plt.plot(df["Deployment"], df["BGP_Updates"], label="BGP Updates")
plt.plot(df["Deployment"], df["Affected_ASes"], label="Affected ASes")
plt.xlabel("ROV Deployment (%)")
plt.ylabel("Count")
plt.title("BGP Metrics vs. ROV Deployment")
plt.legend()
plt.savefig("output.png", dpi=150)
```

Separate scripts handle high-degree, Tier-1, and random experiment sets. Each script follows the same structure, differing only in the input CSV path and plot title.

7. Execution Workflow

The following end-to-end workflow demonstrates a complete experiment cycle using the CLI.

Step	CLI Command	Purpose
1. Load dataset	load dataset = data/caida.txt;	Specify AS topology file
2. Initialize	init;	Build in-memory graph, reset counters
3. Announce prefix	announce 10.0.0.0/8 13335;	Inject BGP route to propagate
4. Simulate	simulate;	Run propagation, collect baseline metrics
5. Enable ROV	rov { enable top100; }	Apply ROV policy block
6. Re-simulate	simulate;	Re-run with ROV, collect post-ROV metrics
7. Export	export results.csv;	Write metrics to CSV for plotting
8. Plot	python plot_results.py results.csv	Generate figures (outside CLI)

Steps 4–7 can be repeated across multiple deployment levels (e.g., top 10, 50, 100 ASes) in a single CLI session without reloading the topology.

8. Design Decisions

- Interactive CLI over static scripts — eliminates the need to restart the engine between parameter changes, reducing experiment iteration time and enabling ad-hoc exploration.
- Lightweight metric hooks — counters are maintained in Python-level module globals, avoiding modifications to the hot C++ propagation path and preserving simulation accuracy.
- Modular experiment scripts — each selection strategy (high-degree, Tier-1, random) is an independent script, making it straightforward to add new strategies or swap datasets.
- Separation of simulation and plotting — decoupling CSV export from visualization means results are archivable and figures can be regenerated or restyled without re-running simulations.
- Temporary file bridge for block execution — reusing `engine.parseDescriptionFile()` for CLI block input avoids duplicating the parser and ensures CLI behaviour matches batch mode exactly.

9. Limitations

- Simulation-level metrics only — convergence time and update counts reflect simulation ticks, not real network latency or hardware timing.
- Depends on predefined invalid-route dataset — ROV experiments require a pre-built dataset of invalid prefixes; generating this from live BGP data is outside the current scope.
- No packet-level modeling — the engine abstracts BGP to route propagation without modeling TCP sessions, NOTIFICATION messages, or hold-timer expiry.
- Limited parallelism — the simulator uses a scheduler with configurable threads, but performance may still be constrained due to synchronization and shared state.
- CLI state is not persisted — closing the CLI discards all loaded parameters and counters; there is no session save/restore mechanism.